



Operating Systems

Assignment # 2

Instructor	Dr. Rana Asif Rehman
Section	BCS 4A + 4B
Teaching Assistant	Muhammad Ali Butt
Semester	Spring 2025

Fast School of Computing

FAST-NU, Lahore, Pakistan

Instructions:

- Submit a single .zip file containing subdirectories for each question.
- Comment your code to keep it readable.
- Plagiarism is not acceptable.

Question No. 1.

In this question, we warm up our pthreads knowledge and syntax.

Create a program that creates an array of size 50,000 containing random numbers. Afterwards, create 2 threads:

- 1st thread will sort the first half of the array using Quick Sort.
- 2nd thread will sort the second half of the array using Insertion Sort.

The main thread should output the time taken by each thread and then save the sorted array into “firstHalf.txt” and “secondHalf.txt”.

Question No. 2.

In this question, we shall demonstrate to what extent can threads accelerate our code.

Write a C program that creates 2 arrays (A and B) of size $n \times n$ filled with random numbers, where n is passed as a command line argument ($n \geq 128$). We have to compute C, which is the product of A and B. Note: Store A and B (and later C) into files (A.txt, B.txt, and C.txt) so that, during the assignment evaluation, it may be verified whether C was correctly calculated.

For a given value of n , we will compute C multiple times, using a different number of threads each time:

- Minimum number of threads: 1.
- Maximum number of threads: 128.
- First, calculate C using 1 thread only (let's call this run 1), then again using 2 threads (run 2), then again using 3 threads (run 3) and so on.
- Each thread will calculate some rows of C.
- In each run, divide the total number of rows of C with the number of threads in that run, so that each thread gets approximately the same amount of work.
- Measure how long it takes us to fully calculate C in each run.
- Now, print out an array of this form:
[(n, #_of_threads_used1, time_taken1), (n, #_of_threads_used2, time_taken2), (n, #_of_threads_used2, time_taken2), ...]
For example, [(512, 1, 10), (512, 2, 5.5), (512, 3, 3), ..., (512, 512, 1)]

- You then have to make a 3D graph (n on the x-axis, # of threads used on the y-axis and the time taken on the z-axis) using the printed array. Helping link is provided.

Do this for n = 128, 256, 512, 1024, 2048. For each n, store the output array which we will now **visualize in 3D!** Go to this [link](#) or as mentioned below> “Open with Google Colaboratory”> paste your arrays in the top cell, then run all 3 cells one-by-one. Once done, download this file (.ipynb format) and upload it with your assignment solution.

Link:

<https://colab.research.google.com/drive/1Na7p2RU2CdXmOUOHhtJvhdQNVFjBafBG?usp=sharing>

Do you observe diminishing returns as we increase the number of threads in any case? Answer inside the code file as a comment.

Question No. 3.

Create a simple game “Red Light, Green Light”. The program will print either “Red Light” or “Green Light” on the console. The player has to press ‘Enter’ on their keyboard when “Green Light” is displayed.

You will use a total of 3 threads (including the Main Thread). One thread will toggle a global variable randomly to indicate Red Light or Green Light. One thread will read this value and print “Red Light” or “Green Light” correspondingly. The Main Thread will take user inputs in an infinite loop and if the player presses ‘Enter’ on their keyboard when the light is green, it will increase their score, otherwise the game is over.

Is it ever possible that the user presses ‘Enter’ when the light is green, yet the game becomes over? Answer inside the code file as a comment.